Informe técnico: **Hecho · Gestor de tareas**

1. Objetivo

Desarrollar una aplicación web interactiva para **crear, editar, eliminar y gestionar tareas**, aplicando programación orientada a objetos, JavaScript moderno, manipulación del DOM, asincronía, consumo de APIs y almacenamiento local, con una interfaz cuidada y responsiva.

2. ¿Qué hace la aplicación?

La pantalla principal es la lista de tareas. Desde ella el usuario puede:

- Agregar o editar una tarea en una ventana modal (descripción, fecha límite, etiqueta y color).
- Completar y eliminar tareas, y buscarlas o filtrarlas por texto, estado o etiqueta.
- Ver alerta de pendientes en la barra superior.
- Seguir una cuenta regresiva de las fechas límite y recibir avisos antes del vencimiento.
- Exportar una tarea a Google Calendar o a un archivo `.ics`.
- Cambiar entre modo claro y oscuro.

**Todo se guarda en el navegador y sobrevive a la recarga de la página (Boton de importar listas para su prueba. Posteriormente, se elimina.

3. Cumplimiento de la consigna

3.1 Programación Orientada a Objetos

Dos clases en `js/app.js`, cada una con una responsabilidad:

| Clase | Contenido |

- Tarea: Propiedades `id`, `descripcion`, `estado`, `fechaCreacion`, `fechaLimite`, `etiqueta`, `color`. Getter `completada` y método `alternarEstado()` que cambia entre *pendiente* y *completada*.
- GestorTareas: Administra la lista: `agregar`, `agregarVarias`, `editar`, `alternar`, `eliminar`, `buscar`, `filtrar` y los getters `pendientes` y `etiquetas`. Los métodos privados `#cargar` y `#guardar` encapsulan `localStorage`. |

Ejemplo de instanciación y uso:

```js >
const tarea = gestor.agregar({ descripcion, fechaLimite, etiqueta, color }); // dentro: new Tarea({ ...datos })

gestor.alternar(tarea.id);   // llama a tarea.alternarEstado() y guarda
```

*Decisión: la eliminación está en `GestorTareas` y no en `Tarea`, porque una tarea no conoce la lista que la contiene; quien administra la colección es el gestor.

3.2 JavaScript ES6+

Característica	Uso en el proyecto
`let` y `const`	`const` por defecto; `let` solo donde el valor cambia (`idEnEdicion`, `desfaseMs`).
Template literals	Mensajes, URLs de la API y textos de la cuenta regresiva
Arrow functions:	Callbacks de `map`, `filter`, `find`, `forEach` y manejadores de eventos
Destructuring	`constructor({ id, descripcion, ... })`, `({ unixtime })` al leer la API, `({ target })` en eventos
Spread	`new Tarea ({...datos })`, `gestor.agregarVarias(...ejemplos)`, `[...new Set(...)]`
Rest	agregarVarias (...lista)` recibe cualquier cantidad de tareas
Valores por defecto	`estado = 'pendiente'`, `filtrar({ estado = 'todas', texto = '' } = {})`
`??` y `?.`	Lectura segura de `localStorage` y de tareas que podrían no existir
Campos privados	• `#clave`, `#cargar()`, `#guardar()`

3.3 Manipulación del DOM y manejo de eventos

Evento	Donde y para qué
`submit` Formulario del modal	valida y crea o edita la tarea (`preventDefault`) |
click`	Un solo listener en la lista atiende completar, editar, eliminar y exportar (**delegación de eventos**); otros para filtros, atajos de fecha, avisos y tema |
`keyup`	| Contador de caracteres de la descripción y búsqueda en vivo
`mouseover`	Muestra la fecha de creación de la tarea bajo el cursor
change`	Filtro por etiqueta

La lista se dibuja clonando un `<template>` de HTML y se inserta con `replaceChildren`, de modo que el DOM se actualiza una sola vez por cambio. El texto del usuario se asigna con `textContent` y no con `innerHTML`, para evitar inyección de HTML.

3.4 Asincronía

Recurso	Uso
`setInterval`	Cada segundo actualiza el reloj, las cuentas regresivas y revisa vencimientos
`setTimeout`	Oculta los avisos tras 3 segundos y muestra la **notificación de resultado 2 segundos después** de guardar una tarea nueva (`RETARDO_AVISO_MS`)
`async/await`	Todas las llamadas a la API, escritas de forma secuencial y legible
`clearTimeout`	Antes de reiniciar un aviso se usa para que dos temporizadores no se pisen.

3.5 Consumo de APIs y almacenamiento

fetch` con manejo de errores.** Toda petición pasa por `pedirJson`, que revisa `respuesta.ok` (porque `fetch` no falla por sí solo ante un error HTTP) y lanza un error si corresponde.

Función	Petición
`obtenerEjemplos`	GET** a JSONPlaceholder: el botón *Importar ejemplos* trae 5 tareas y las convierte al formato de la app
`enviarTarea`	**POST** a JSONPlaceholder: cada tarea nueva se envía a la API
`sincronizarHora`	**GET** a WorldTimeAPI: ajusta el reloj de Chile

Todas están dentro de `try/catch`; la importación usa además `finally` para reactivar el botón. Si

una API falla, el usuario recibe un aviso o la app usa un respaldo (por ejemplo, la hora del dispositivo), y sigue funcionando.

El localStorage` en `GestorTareas` guarda la lista como JSON en cada cambio y la recupera al iniciar; la lectura está protegida con `try/catch` por si los datos estuvieran dañados. También se guarda la preferencia de tema. JSONPlaceholder acepta peticiones, pero no almacena datos, por eso `localStorage` es la *fuente real* de las tareas y la API demuestra el consumo y manejo de errores.

4. Funcionalidades adicionales

Función	Aporte	Implementación
Hora y fecha de Chile	Orienta al fijar plazos; la fecha límite no puede ser pasada	`Intl.DateTimeFormat` con zona `America/Santiago`
Colores y etiquetas	Diferenciar y agrupar tareas	Propiedades `color` y `etiqueta` en `Tarea`; filtro por etiqueta
Exportar a calendario	Llevar la tarea a Google Calendar o a un `.ics`	`URLSearchParams` y `Blob`
Avisos	Alertar antes del vencimiento	Aviso en página y API `Notification` del navegador
Modo oscuro	Comodidad visual	`data-bs-theme` de Bootstrap y variables CSS

5. Decisiones técnicas

- Bootstrap por CDN**, porque se trabaja en el curso y resuelve la grilla, el modal y el modo oscuro sin código propio. El CSS propio queda limitado a la identidad visual y al estado de las tareas.
- Un único `app.js` sin módulos, para que el proyecto abra con doble clic.
- Delegación de eventos: un listener en la lista funciona, aunque esta se redibuje.
- Fechas contra la hora de Chile, no contra la del dispositivo, para que el resultado sea el mismo, aunque el equipo tenga otra zona horaria.
- Archivo `.ics` con hora "flotante": el calendario la interpreta en la zona del dispositivo. Mantiene el código simple.

6. Buenas prácticas

- Separación de responsabilidades: HTML (estructura), CSS (presentación) y JS (comportamiento) en archivos distintos; dentro de JS, tres bloques: modelo, API e interfaz.

- HTML semántico y accesible: `nav`, `main`, `section`, `label` asociado a cada campo, `aria-live`, `aria-label` en botones de ícono, foco visible.

- Código limpio: funciones pequeñas con un propósito, sin repetición (una sola función `pedirJson` para toda petición y una sola `renderizar`), modo estricto y comentarios.

- Seguridad básica: `textContent`, validación del formulario antes de crear la tarea.

- Diseño responsivo (mobile-first): grilla de Bootstrap, unidades relativas, y `prefers-reduced-motion` respetado.

7. ¿Cómo probar la app?

Para comprobar el correcto funcionamiento de la aplicación, realiza las siguientes pruebas en orden:

1. Crear una tarea

Completa los campos correspondientes y guarda una nueva tarea utilizando el formulario.

Se verifica: Programación Orientada a Objetos (POO), evento submit y manipulación del DOM.

2. Probar la escritura y búsqueda

Escribe texto en el campo de descripción de una tarea y utiliza el buscador para filtrar las tareas.

Se verifica: Evento keyup, permitiendo detectar los cambios realizados mientras el usuario escribe.

3. Pasar el cursor sobre una tarea

Ubica el cursor del mouse sobre una tarea existente y observa la respuesta visual o funcional de la aplicación.

Se verifica: Evento mouseover.

4. Crear una tarea con fecha límite cercana

Crea una tarea cuya fecha límite esté próxima y observa cómo se actualiza la cuenta regresiva.

Se verifica: Uso de setInterval para actualizar el contador de tiempo restante.

5. Revisar el aviso después de guardar

Guarda una tarea y espera aproximadamente 2 segundos. Debería aparecer el aviso correspondiente.

Se verifica: Uso de setTimeout y envío de información mediante fetch con método POST.

6. Importar tareas de ejemplo

Pulsa el botón Importar ejemplos y verifica que las tareas de ejemplo se incorporen correctamente a la aplicación.

Se verifica: Consumo de una API mediante fetch con método GET, además del uso de operadores rest y spread.

7. Recargar la página

Después de crear o importar tareas, recarga la página y comprueba que la información permanezca disponible.

Se verifica: Persistencia de datos mediante localStorage.

8. Probar el funcionamiento sin conexión

Desconecta temporalmente la conexión a internet y pulsa Importar ejemplos. La aplicación debería manejar el error sin detenerse ni romper su funcionamiento.

Se verifica: Manejo de errores mediante try/catch.

9. Abrir el menú de calendario

En una tarea, abre el menú asociado al calendario y comprueba las opciones disponibles para trabajar con la información de la tarea.

Se verifica: Funcionalidad de exportación de datos.

Resultado esperado

Al completar estas pruebas, se puede comprobar el funcionamiento de las principales características de la aplicación y la implementación de los conceptos de JavaScript solicitados: POO, eventos del DOM, programación asíncrona, fetch, manejo de errores, localStorage y exportación de datos.